

Topic 33 — INST Format: LLaMA-2 Instruction Specification

Full Stack Python + AI Roadmap • Phase 4: Prompt Serialization & Instruction Formats (final topic) • 3 layers

Layer 1 — New to the Concept

INST format is the prompt serialization style used by Meta's **LLaMA-2** chat models. "INST" is short for "instruction," and the format wraps the user's instruction in **tags**:

```
[INST] What is the capital of France? [/INST]
```

`[INST]` opens the instruction, `[/INST]` closes it — like an HTML tag pair (`<b>...</b>`). Everything between the tags is the user's message; whatever the model generates *after* `[/INST]` is its response.

The three styles you've now seen, side by side:

Alpaca	→	### Instruction: ... ### Response:	(text headers)
ChatML	→	< im_start >user ... < im_end >	(special-token markers)
INST	→	[INST] ... [/INST]	(bracket tags)

Same job — mark where the user's turn is and where the model responds — three different notations. INST is LLaMA-2's.

Layer 2 — The Full Format

Basic single-turn form

```
<s>[INST] Explain recursion in one sentence. [/INST]
```

- `<s>` — a special "beginning of sequence" (BOS) token marking the start. (A matching `</s>` end-of-sequence token is used between turns.)
- The model writes its answer right after `[/INST]`.

Adding a system prompt — the `<<SYS>>` tags

```
<s>[INST] <<SYS>>
You are a helpful assistant. Always answer concisely.
<</SYS>>

Explain recursion in one sentence. [/INST]
```

The system prompt goes between `<<SYS>>` and `</SYS>>`, *inside* the first instruction block, followed by the actual user message. Different from ChatML (where system is its own separate turn) — LLaMA-2 nests it into the first user turn.

Multi-turn — stacking with `<s>` and `</s>`

```
<s>[INST] What's 2+2? [/INST] 4 </s><s>[INST] And times 3? [/INST]
```

Each full exchange is wrapped: `<s>[INST] user [/INST] assistant </s>`, and the next turn starts a fresh `<s>[INST]...`. The previous answer (`4`) stays as history. More intricate than ChatML's clean repeating blocks — the `<s>` / `</s>` boundaries and nesting make INST fiddlier to hand-write.

 Layer 3 — Expert (details, pitfalls & big picture)

1. Tags and BOS/EOS are special tokens

`[INST]`, `[/INST]`, `<<SYS>>`, `<s>`, `</s>` are meaningful because LLaMA-2 was trained on exactly this structure. `<s>` (BOS) and `</s>` (EOS) are genuine special tokens; the model uses `</s>` to learn where a turn ends. Getting these subtly wrong is a common source of poor LLaMA-2 output.

2. Whitespace is famously finicky

⚠ Notoriously sensitive: the exact spaces around `[INST]` and newlines around `<<SYS>>` matter — mismatches measurably hurt output. This is the poster child for *why you shouldn't hand-format*.

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")

messages = [
    {"role": "system", "content": "You are helpful."},
    {"role": "user", "content": "Explain recursion."}
]
prompt = tokenizer.apply_chat_template(messages, tokenize=False)
# → correct "<s>[INST] <<SYS>>...</SYS>>\n\nExplain recursion. [/INST]"
```

`apply_chat_template()` (Topic 30) handles all the `<s>`, `[INST]`, `<<SYS>>`, and whitespace details. For LLaMA-2 especially, strongly preferred over manual string-building.

3. The three formats — complete comparison

Feature	Alpaca	ChatML	INST (LLaMA-2)
Marker style	### headers	< im_start > / < im_end >	[INST] / [/INST]
System prompt	Preamble sentence	Separate <code>system</code> turn	Nested <code><<SYS>></code> in first <code>[INST]</code>

Multi-turn	No	Yes (clean blocks)	Yes (<code><s> . . . </s></code> wrapped)
BOS/EOS	None	Implicit	Explicit <code><s></code> / <code></s></code>
Origin	Stanford Alpaca	OpenAI	Meta LLaMA-2
Hand-writing	Easy	Moderate	Fiddly (whitespace)

This table is the whole point of Phase 4: same conceptual prompt, three concrete serializations, each tied to a model family.

4. Version churn — a caution

i **Formats change:** LLaMA-3 moved away from INST to a new format (header tokens like `<|start_header_id|>`), closer to ChatML in spirit. Mistral uses a variant of `[INST]`. INST is specifically a LLaMA-2-era format — the lesson isn't to memorize one set of tags forever, but to *always check the model card* and use its chat template.

5. Why this whole phase matters

A prompt isn't just "text you send" — it's text in a **specific serialized structure** the target model expects. With hosted APIs this is invisible; with open-source models (LLaMA, Mistral) it's your responsibility, and getting it right is a real, silent quality lever. Universal answer: use `apply_chat_template()` and check the model card, don't hand-format.

6. Practical relevance

✓ For your work: deploying open-source models (cost savings, data control, self-hosting for MSG2AI), you'll pick LLaMA or Mistral and serialize prompts in their expected format. Knowing INST (and that LLaMA-3+ differs) means you'll recognize formats in model docs, debug format-related quality issues, and lean on chat templates rather than fighting whitespace by hand.

 Interview Q&A Cards

Q: What is the INST format?

A: LLaMA-2's instruction format: the user turn wrapped in `[INST] . . . [/INST]` tags, the system prompt nested inside the first block via `<<SYS>> . . . </SYS>>`, and turns bounded by `<s>` / `</s>` BOS/EOS tokens.

Q: How does INST handle the system prompt vs ChatML?

A: INST nests the system prompt inside the first `[INST]` block using `<<SYS>>` tags, whereas ChatML gives system its own separate turn. Different placement, same intent.

Q: Why prefer a chat template over hand-formatting INST?

A: INST is highly whitespace-sensitive and uses several special tokens; small mistakes silently degrade output. `apply_chat_template()` produces the exact correct string, avoiding those errors.

Q: Is INST universal across LLaMA models?

A: No — it's LLaMA-2-specific. LLaMA-3 uses a different header-token format and Mistral uses an INST variant. Always check the model card and use its template rather than assuming.

🔗 **Memory hook:** *INST = LLaMA-2's `[INST] ... [/INST]` tags. System prompt nested in `<<SYS>>...` inside the first block. Turns wrapped in `<s>...</s>` (BOS/EOS). Whitespace-finicky → use `apply_chat_template()`. LLaMA-2-specific (LLaMA-3 differs). Three styles: Alpaca headers, ChatML markers, INST tags.*

✅ **Phase 4 complete!** All 4 Prompt Serialization & Instruction Formats topics done: Intro to Serialization Styles, Alpaca Template, ChatML Schema, and INST Format. You can now recognize and produce the three major prompt serialization styles and know to rely on chat templates.